

{mid-term} [presentation]

Dokumentation | Dominique-Claire Bruttel

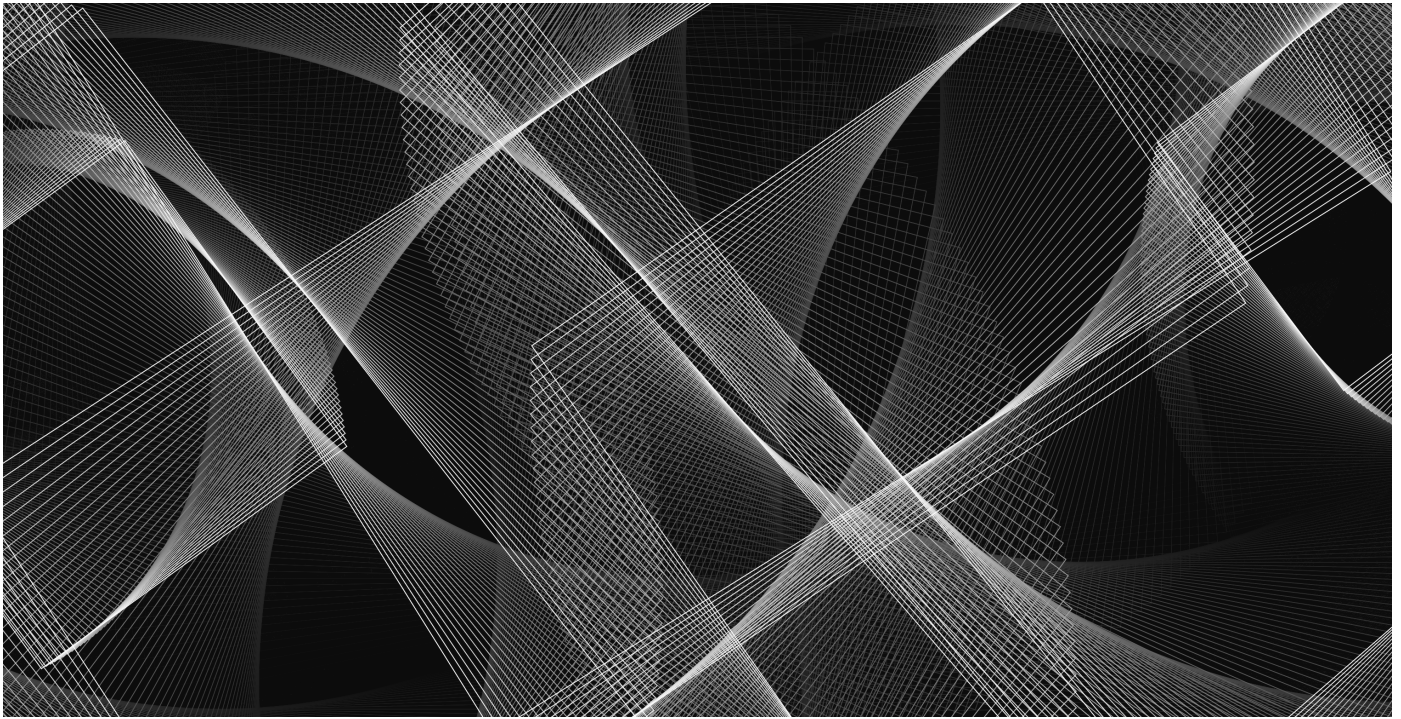
★ DREHENDE RECHTECKE

```
function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  // leicht transparenter HG > Spuren
  // >> Spuren (5, 50, 200) / HG / Kontur
  background(0, 10)
  stroke(255)
  noFill()
  // zeichnet Rechteck von Mitte aus
  rectMode(CENTER)

  // Anz Rechtecke
  let lc = 10;
  // for-Loop > zeichnet mehrere Rechtecke
  for(let i = 0; i < lc; i++) {
    // X-Pos durch Sinus > horizontale Bewegung
    // >> frameCount * X -> Geschwindigkeit (0.01 / 0.05)
    // >> width / x -> Bewegungsgeschwindigkeit (1, 10)
    let x = sin(i * 3.4 + frameCount * .001) * width / 2;
    // Y-Pos durch Cosinus > vertikale Bewegung
    let y = cos(i * 9 + frameCount * .005) * height / 2;
    // Grösse der Rechtecke > an Fensterbreite anpassen
    let s = sin(i * .5 + frameCount * .0012) * width;

    // Transformation starten!
    push()
    // Verschiebt Position von Ursprung
    translate(width / 2 + x, height / 2 + y, s)
    // Dreht Rechteck
    // >> frameCount / 2 -> Geschwindigkeit
    rotate(radians(i + 5 + frameCount / 1))
    rect(0,0,s)
    // // Transformation beenden!
    pop()
  }
}
```



★ TYPOGRAFIE BEEINFLUSST DURCH AUDIO

```
// — Externe Bibliotheken (werden von p5Live automatisch geladen) —————
// hydra-synth: Echtzeit-Shader/Videosynthese im Browser
// hy5.js:      Brücke zwischen p5.js und Hydra (stellt H, s0, P5 bereit)
let libs = [
  'https://unpkg.com/hydra-synth',
  'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js'
]

// — Hydra-Setup (läuft einmalig beim Laden) —————
H.pixelDensity(2) // Retina-Auflösung; auf <= 1 setzen falls es ruckelt
s0.initP5()       // p5-Canvas als Quelle s0 an Hydra übergeben
P5.toggle(0)      // p5-Canvas verstecken — nur Hydra-Output sichtbar

// Hydra-Pipeline:
// src(s0)          → p5-Canvas als Eingangsbild nehmen
// .modulate(noize(4)) → Pixel-Positionen mit organischem Rauschen verschieben
//                  (4 = Rauschfrequenz; höher = feiner, kleiner = gröber)
// .out()           → Ergebnis auf den Hydra-Output rendern
src(s0)
  .modulate(noize(4))
  .out()

// — Globale Audio-Variablen —————
let mic, fft, amplitude
let micReady = false // Schutzflag: Audio erst nutzen wenn Mikrofon bereit

let speed = 1 // Aktuelle (geglättete) Animationsgeschwindigkeit
let speedTarget = 1 // Zielwert direkt aus der Lautstärke

// Dynamische Min/Max-Werte je Frequenzband für Auto-Normalisierung:
// Das Programm lernt mit der Zeit, was in dieser Umgebung "laut" und "leise" ist
let energyMin = { bass: 255, mid: 255, treble: 255 }
```

```

let energyMax = { bass: 0, mid: 0, treble: 0 }

// — setup(): einmalig beim Start —————
function setup() {
  createCanvas(windowWidth, windowHeight)
  angleMode(DEGREES) // sin() erwartet Grad, nicht Bogenmass
  textSize(width * 0.01) // Textgrösse = 1% der Canvas-Breite
  textAlign(CENTER, CENTER)

  mic = new p5.AudioIn()
  mic.start(() => { // Callback: wird aufgerufen sobald Browser-Erlaubnis da
    fft = new p5.FFT(0.8, 1024) // 0.8 = Glättung, 1024 = Frequenzauflösung
    fft.setInput(mic)
    amplitude = new p5.Amplitude(0.9) // 0.9 = Glättung der Lautstärkemessung
    amplitude.setInput(mic)
    micReady = true
  })
}

// — draw(): läuft jeden Frame (~60×/Sek.) —————
function draw() {
  background(0) // Vorherigen Frame löschen

  if (micReady) {
    fft.analyze() // FFT-Puffer aktualisieren – muss jeden Frame aufgerufen werden

    // Lautstärke → Zielgeschwindigkeit: leise = 0.1, laut = 6
    speedTarget = map(amplitude.getLevel(), 0, 0.3, 0.1, 6, true)
  }

  // Geschwindigkeit weich interpolieren → keine abrupten Sprünge
  // 0.08 = Trägheit: kleiner = langsamer/weicher, grösser = direkter
  speed = lerp(speed, speedTarget, 0.08)

  // — Hilfsfunktion: Frequenzenergie einer Spalte → Helligkeit (0–255) —
  // Hinweis: hier innerhalb draw() definiert – wird jedes Frame neu erstellt,
  // funktioniert aber, da JS Funktionen als Werte behandelt
  function getFreqColor(colIndex, totalCols) {
    if (!micReady) return 128 // Grau als Fallback vor Mikrofon-Start

    // t: Position der Spalte im Spektrum (0.0 = ganz links, 1.0 = ganz rechts)
    let t = colIndex / (totalCols - 1)

    let bass = fft.getEnergy("bass") // Tiefton-Energie (0–255)
    let mid = fft.getEnergy("mid") // Mittelton-Energie
    let treble = fft.getEnergy("treble") // Hochton-Energie

    // Min/Max dynamisch nachführen:
    // Min: sinkt sehr langsam (0.01) → passt sich träge an ruhige Phasen an
    // Max: steigt schneller (0.05) → reagiert flott auf neue Spitzenwerte
    energyMin.bass = lerp(energyMin.bass, min(energyMin.bass, bass), 0.01)
    energyMin.mid = lerp(energyMin.mid, min(energyMin.mid, mid), 0.01)
    energyMin.treble = lerp(energyMin.treble, min(energyMin.treble, treble), 0.01)
    energyMax.bass = lerp(energyMax.bass, max(energyMax.bass, bass), 0.05)
    energyMax.mid = lerp(energyMax.mid, max(energyMax.mid, mid), 0.05)
    energyMax.treble = lerp(energyMax.treble, max(energyMax.treble, treble), 0.05)

    // Rohwerte relativ zum beobachteten Min/Max auf 0–255 normalisieren

```

```

// → Auto-Gain: schwache Signale werden genauso ausgenutzt wie starke
let bN = map(bass,    energyMin.bass,    energyMax.bass,    0, 255, true)
let mN = map(mid,    energyMin.mid,    energyMax.mid,    0, 255, true)
let tN = map(treble, energyMin.treble, energyMax.treble, 0, 255, true)

// Spaltenposition bestimmt die Frequenzmischung:
// linke Hälfte (t < 0.5): Bass → Mid
// rechte Hälfte (t ≥ 0.5): Mid → Treble
let energy = t < 0.5 ? lerp(bN, mN, t * 2) : lerp(mN, tN, (t - 0.5) * 2)

// >> Farbausgabe (1. s/w | unteres 2. violet-schwarz)
// return constrain(energy, 0, 255)
// Neu (Violett → Schwarz):
let e = constrain(energy, 0, 255)
return [map(e, 0, 255, 0, 180), 0, map(e, 0, 255, 0, 255)]

}

// — Hilfsfunktion: eine Spalte schwingende Textzeilen zeichnen —————
// modulation: Wellenversatz dieser Spalte (in Grad)
// posX:      x-Position der Spalte
// brightness: Füllhelligkeit aus Frequenzanalyse (0–255)

// Farbausgabe (1. s/w | unteres 2. violet-schwarz)
//function funkyText(modulation, posX, brightness) {
//  fill(brightness)
//  function funkyText(modulation, posX, col) {
fill(col[0], col[1], col[2])

  for (let i = 0; i < height / 20; i++) { // Zeilen alle 20px von oben nach unten
    // Sinuswelle für seitliche Auslenkung:
    // frameCount * 0.1 * speed → Animationsgeschwindigkeit
    // i * modulation          → jede Zeile hat einen anderen Wellenversatz
    let offset = sin(frameCount * 0.1 * speed + (i * modulation))

    text(
      "D'n'B",
      posX + offset * width * 0.05, // Auslenkung: maximal ±5% der Canvas-Breite
      i * 20
    )
  }
}

// — Spalten zeichnen —————
let columns = 10 // Anzahl Textspalten (vorher 8, jetzt 10)

for (let i = 0; i < columns; i++) {
  // Spalten gleichmässig zwischen 10% und 90% der Canvas-Breite
  let posX = map(i, 0, columns - 1, width * 0.1, width * 0.9)

  funkyText(
    30 + i * 20, // Wellenversatz: steigt pro Spalte um 20°
    posX,
    getFreqColor(i, columns) // Helligkeit aus dem Frequenzspektrum
  )
}
}

```

```
// — windowResized(): Canvas bei Fenstergrößen-Änderung anpassen —
function windowResized() {
  resizeCanvas(windowWidth, windowHeight)
}
```

★ FLOCK (BEWEGENDE DREIECKE, GRUPPIEREN UND FOLGEN SICH)

```
// noprotect
let flock;

function setup() {
  createCanvas(windowWidth, windowHeight);
  flock = new Flock();
  for (let i = 0; i < 100; i++) flock.add(new Boid(width/2, height/2));
}
function draw() { background(0); flock.run(); }
function mouseDragged() { flock.add(new Boid(mouseX, mouseY)); }

// — Flock —
class Flock {
  constructor() { this.boids = []; }
  add(b) { this.boids.push(b); }
  run() { this.boids.forEach(b => b.run(this.boids)); }
}

// — Boid —
class Boid {
  constructor(x, y) {
    this.pos = createVector(x, y);
    this.vel = createVector(random(-1,1), random(-1,1));
    this.acc = createVector(0, 0);
    this.r = 3; this.ms = 3; this.mf = 0.05;
  }

  run(bs) {
    this.acc.add(this.separate(bs).mult(1.5));
    this.acc.add(this.align(bs));
    this.acc.add(this.cohesion(bs));
    this.vel.add(this.acc).limit(this.ms);
    this.pos.add(this.vel);
    this.acc.set(0, 0);
    this.borders();
    this.render();
  }

  // Gemeinsame Logik für align + cohesion + separate
}
```

```

steer(bs, dist, fn) {
  let sum = createVector(0, 0), count = 0;
  for (let o of bs) {
    let d = p5.Vector.dist(this.pos, o.pos);
    if (d > 0 && d < dist) { fn(sum, o, d); count++; }
  }
  return count > 0 ? sum.div(count) : createVector(0, 0);
}

// Gemeinsame Normalisierung für align + cohesion
limit(s) {
  return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
}

seek(t) {
  return this.limit(p5.Vector.sub(t, this.pos));
}

separate(bs) {
  let s = this.steer(bs, 25, (sum, o, d) =>
    sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
  return s.mag() > 0 ? this.limit(s) : s;
}

align(bs) {
  let s = this.steer(bs, 50, (sum, o) => sum.add(o.vel));
  return s.mag() > 0 ? this.limit(s) : s;
}

cohesion(bs) {
  let s = this.steer(bs, 50, (sum, o) => sum.add(o.pos));
  return s.mag() > 0 ? this.seek(s) : s;
}

// Wiederholung für x und y zusammengefasst
borders() {
  for (let [ax, max] of [['x', width], ['y', height]]) {
    if (this.pos[ax] < -this.r) this.pos[ax] = max + this.r;
    if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r;
  }
}

render() {
  push();
  translate(this.pos.x, this.pos.y);
  rotate(this.vel.heading() + radians(90));
  fill(255, 217, 249);
  stroke(252, 121, 230);
  // noStroke();
  beginShape();
  vertex( 0, -this.r * 2);
  vertex(-this.r, this.r * 2);
  vertex( this.r, this.r * 2);
  endShape(CLOSE);
  // circle(0, 0, this.r * 4);
  pop();
}
}

```

